Software Engineer Assignment

Recommendation Service System Design and Implementation

1. Overview

This assignment evaluates your ability to design and implement a production-ready backend recommendation service. You will demonstrate proficiency in system architecture, database design, concurrency management, caching strategies, error handling, and performance optimization.

Expected Time Commitment: 4-6 hours

2. System Workflow Overview

Understanding the complete request flow is crucial for implementing this system correctly. The following sections describe the workflow for both single and batch recommendation requests.

2.1 Single User Recommendation Workflow

Request Flow:

1. **Client Request:** HTTP `GET /users/{user_id}/recommendations` arrives at the Handler layer
2. **Input Validation:** Handler validates `user_id` and query parameters (`limit`)
3. **Cache Check:** Service layer queries Redis cache using key pattern: `rec:user:{user_id}:limit:{limit}`
4. **Cache Hit Path:** If found, deserialize and return cached recommendations (`metadata.cache_hit = true`)
5. **Cache Miss Path:** Proceed to generate fresh recommendations:
   - **Fetch User Data:** Repository queries `users` table for user profile
   - **Fetch Watch History:** Repository queries `user_watch_history` with JOIN
   - **Fetch Candidate Content:** Repository queries `content` table, excluding watched items
   - **Model Inference:** Model Client applies scoring algorithm
   - **Rank and Filter:** Sort by score descending, take top N items
6. **Cache Store:** Serialize recommendations and store in Redis with TTL (default: 10 minutes)
7. **Response:** Format and return JSON response with recommendations and metadata

2.2 Batch Recommendation Workflow

Request Flow:

1. **Client Request:** HTTP `GET /recommendations/batch?page=1&limit=20` arrives at Handler
2. **Parameter Validation:** Validate `page` and `limit` parameters
3. **User ID Fetching:** Repository executes paginated query
4. **Concurrent Processing:** Service layer creates bounded worker pool (you decide optimal concurrency)
5. **Per-User Processing:** For each user, spawn async task
6. **Result Aggregation:** Collect all results (successful and failed)
7. **Summary Calculation:** Count success/failure, measure processing time

8. **Response:** Return batch response with pagination info and summary

2.3 Critical Implementation Notes

- **Avoid N+1 Queries:** Prefetch user data and watch histories in bulk
- **Connection Pool Management:** Configure appropriate database connection pool size
- **Timeout Handling:** Set reasonable timeouts to prevent hanging requests
- **Cache Invalidation:** Clear cache when user watch history is updated

---

3. Technical Stack Requirements

- **Programming Language:** Choose one: Go, Rust, Java, Python, Node.js, or equivalent
- **Database:** PostgreSQL (preferred) or SQLite
- **Cache Layer:** Redis
- **Containerization:** Docker and Docker Compose
- **Performance Testing:** k6 load testing framework

---

4. Database Schema Design

Implement the following normalized relational schema:

4.1 Users Table

```
CREATE TABLE users (
    id BIGSERIAL PRIMARY KEY,
    age INT NOT NULL CHECK (age >0),
    country VARCHAR(2) NOT NULL,
    subscription_type VARCHAR(20) NOT NULL,
    created_at TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_users_country ON users(country);
CREATE INDEX idx_users_subscription ON users(subscription_type);
```

4.2 Content Table

```
CREATE TABLE content (
    id BIGSERIAL PRIMARY KEY,
    title VARCHAR(255) NOT NULL,
    genre VARCHAR(50) NOT NULL,
    popularity_score DOUBLE PRECISION NOT NULL CHECK (popularity_score >= 0),
    created_at TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_content_genre ON content(genre);
CREATE INDEX idx_content_popularity ON content(popularity_score DESC);
```

4.3 User Watch History Table

```
CREATE TABLE user_watch_history(
    id BIGSERIAL PRIMARY KEY,
    user_id BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    content_id BIGINT NOT NULL REFERENCES content(id) ON DELETE CASCADE,
    watched_at TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_watch_history_user ON user_watch_history(user_id);
CREATE INDEX idx_watch_history_content ON user_watch_history(content_id);
CREATE INDEX idx_watch_history_composite ON user_watch_history(user_id, watched_at DESC);
```

4.4 Index Strategy Explanation

Index	Purpose
`idx_watch_history_user`	Speeds up fetching watch history for a specific user
`idx_watch_history_composite`	Optimizes queries that need recent watch history ordered by time
`idx_content_genre`	Enables fast filtering by genre for recommendation candidates
`idx_content_popularity`	DESC ordering helps quickly fetch top popular content

5. Data Seeding Requirements

Implement a deterministic seeding mechanism with the following specifications:

5.1 Minimum Dataset Size

- **Users:** Minimum 20 records
- **Content Items:** Minimum 50 records
- **Watch History Records:** Minimum 200 records
- **Genre Diversity:** Minimum 5 distinct genres (e.g., action, drama, comedy, thriller, documentary)

5.2 Data Distribution Requirements

- **Popularity Distribution:** Must follow a realistic non-uniform distribution (e.g., power law or long-tail distribution)
- **Subscription Types:** Include varied subscription tiers (e.g., free, basic, premium)
- **Geographic Distribution:** Include users from multiple countries (use ISO 3166-1 alpha-2 codes)
- **Deterministic Generation:** Use a fixed seed to ensure reproducible results across runs

6. System Architecture Requirements

6.1 Layered Architecture

Implement clear separation of concerns using the following architectural layers:

```
┌──────────────────────────────────────────┐
│          Handler Layer                   │
│  (HTTP, Validation, Serialization)       │
└──────────────┬───────────────────────────┘
               │
┌──────────────▼───────────────────────────┐
│          Service Layer                   │
│  (Business Logic, Orchestration)         │
└──────┬──────────────────────────┬────────┘
       │                          │
┌──────▼──────────┐      ┌────────▼────────┐
│  Repository     │      │  Model Client   │
│   (Data Access) │      │   (Scoring)     │
└──────┬──────────┘      └─────────────────┘
       │                          │
┌──────▼──────────────────────────▼────────┐
│     Database + Cache Layer               │
│     (PostgreSQL + Redis)                 │
└──────────────────────────────────────────┘
```

Layer Responsibilities:

- **Handler Layer:** HTTP request/response handling, input validation, response formatting
- **Service Layer:** Business logic, orchestration, error handling, workflow coordination
- **Repository Layer:** Data access abstraction, query implementation, database operations
- **Model Client:** Recommendation algorithm, scoring logic (simulated), feature engineering
- **Cache Layer:** Redis integration, cache key management, TTL handling

6.2 Code Organization

- No monolithic single-file implementations
- Follow language-specific best practices for project structure
- Maintain clean, readable, and well-documented code
- Use dependency injection where appropriate

---

7. Recommendation Model Specification and Database Integration

Since this is a backend engineering assessment, you will **simulate** a machine learning model rather than implement actual ML algorithms. However, the simulation must demonstrate understanding of how production ML systems integrate with databases and produce meaningful, non-random recommendations.

7.1 Model Input Data Requirements

The Model Client must receive the following data from the database through the Repository layer:

User Context (from `users` table):

- User ID (for cache key generation)
- Age (can influence content appropriateness)
- Country (for geo-specific content preferences)
- Subscription Type (affects content access levels)

Watch History (from `user_watch_history` JOIN `content`):

```
SELECT c.id, c.genre,uwh.watched_at
FROM user_watch_history uwh
JOIN content c ON uwh.content_id = c.id
WHERE uwh.user_id = ?
ORDER BY uwh.watched_at DESC
LIMIT 50;
```

Derived Data:

- List of content IDs already watched (for exclusion)
- Genre distribution (e.g., `{action: 10, drama: 5, comedy: 3}`)
- Most recent watch timestamps

Candidate Content (from `content` table):

```
SELECT id, title, genre, popularity_score, created_at
FROM content
WHERE id NOT IN (
    SELECT content_id
    FROM user_watch_history
    WHERE user_id = ?
)
ORDER BY popularity_score DESC
LIMIT 100;
```

7.2 Scoring Algorithm Implementation

The Model Client must implement a **heuristic-based scoring function** that uses the database data to compute meaningful recommendations.

Step 1: Calculate Genre Preference Weights

```
# Build genre preference map from watch history
genre_counts = {} # e.g., {"action": 10, "drama": 5, "comedy": 3}
total_watches = sum(genre_counts.values()) # 18

# Normalize to get preference scores (0.0 - 1.0)
genre_preferences = {
    genre: count / total_watches
    for genre, count in genre_counts.items()
}
# Result: {"action": 0.556, "drama": 0.278, "comedy": 0.167}
```

Step 2: Calculate Recency Factor

```
# Time decay: newer content gets slight boost
current_time = now()
days_since_creation = (current_time - content.created_at).days
recency_factor = 1.0 / (1.0 + days_since_creation / 365.0)

# Examples:
# - Content from 1 year ago: recency_factor ≈ 0.5
# - Content from 1 week ago: recency_factor ≈ 0.98
```

Step 3: Compute Final Score

```
for candidate_content in candidates:
    # Base component: content popularity
    popularity_component = content.popularity_score * 0.4

    # Genre match component: prefer genres user has watched
    genre_boost = genre_preferences.get(content.genre, 0.1) * 0.35

    # Recency component: slight preference for newer content
    recency_component = recency_factor * 0.15

    # Exploration component: controlled randomness
    random_noise = random.uniform(-0.05, 0.05) * 0.1

    # Final score
    final_score = (
        popularity_component +
        genre_boost +
        recency_component +
        random_noise
    )

    candidate_content.score = final_score
```

7.3 Simulation Requirements

- **Realistic Latency:** Introduce 30-50ms delay per recommendation request
- **Failure Simulation:** Randomly fail 1-2% of requests
- **Non-Deterministic Elements:** Include controlled randomness

7.4 Content Filtering Rules

- **Exclusion:** Never recommend content the user has already watched
- **Subscription Tier:** (Optional) Filter content based on user's subscription type
- **Geographic Availability:** (Optional) Apply country-based content restrictions

7.5 Integration Example Flow

Complete data flow from database to recommendations:

```
def generate_recommendations(user_id, limit):
    """
    Generate personalized recommendations for a user.

    Args:
        user_id: User identifier
        limit: Maximum number of recommendations to return

    Returns:
        List of scored recommendations
    """
    # 1. Fetch user data from database
    user =repository.get_user_by_id(user_id)
    if not user:
        raise UserNotFoundException(f"User {user_id} not found")

    # 2. Fetch watch history with genres
    watch_history=repository.get_user_watch_history_with_genres(user_id)
    watched_content_ids = [item.content_id for item in watch_history]
    genre_counts = count_genres(watch_history)

    # 3. Calculate genre preferences
    genre_preferences = normalize_genre_counts(genre_counts)

    # 4. Fetch candidate content (excluding watched)
    candidates = repository.get_unwatched_content(user_id, limit=100)

    # 5. Simulate model latency
    time.sleep(random.randint(30, 50) / 1000.0) # 30-50ms

    # 6. Simulate random failure
    if random.random() < 0.015: # 1.5% failure rate
        raise ModelInferenceException("Model inference failed")

    # 7. Score each candidate
    scored_candidates = []
    for content in candidates:
        score = calculate_score(content, genre_preferences, user)
        scored_candidates.append({
            "content_id": content.id,
            "title": content.title,
            "genre": content.genre,
            "popularity_score": content.popularity_score,
            "score": score
        })

    # 8. Sort and return top N
    scored_candidates.sort(key=lambda x: x["score"], reverse=True)
    return scored_candidates[:limit]
```

---

8. API Specifications

8.1 Single User Recommendation Endpoint

Endpoint: `GET /users/{user_id}/recommendations`

Description: Retrieves personalized content recommendations for a specific user based on their watch history, preferences, and content popularity.

Path Parameters

Parameter	Type	Required	Description
`user_id`	integer	required	Unique identifier of the user

Query Parameters

Parameter	Type	Required	Description
`limit`	integer	optional	Number of recommendations to return (default: 10, max: 50)

Response Format (200 OK)

```
{
  "user_id": 1,
  "recommendations": [
    {
      "content_id": 10,
      "title": "Action Movie A",
      "genre": "action",
      "popularity_score": 0.87,
      "score": 1.04
    }
  ],
  "metadata": {
    "cache_hit": false,
    "generated_at": "2026-02-15T10:30:00Z",
    "total_count": 1
  }
}
```

Response Schema

Field	Type	Description
`user_id`	integer	User identifier from request
`recommendations`	array	List of recommended content items, ordered by score (descending)
`recommendations[].content_id`	integer	Unique identifier of the content
`recommendations[].title`	string	Title of the content
`recommendations[].genre`	string	Genre category
`recommendations[].popularity_score`	float	Content popularity metric (0.0 - 1.0)
`recommendations[].score`	float	Final recommendation score (higher is better)
`metadata.cache_hit`	boolean	Indicates if result was served from cache
`metadata.generated_at`	string (ISO 8601)	Timestamp when recommendations were generated
`metadata.total_count`	integer	Number of recommendations returned

Error Responses

Status Code	Response Body
404 Not Found	`{"error": "user_not_found", "message": "User with ID {user_id} does not exist"}`
400 Bad Request	`{"error": "invalid_parameter", "message": "Invalid limit parameter"}`
500 Internal Server Error	`{"error": "internal_error", "message": "An unexpected error occurred"}`
503 Service Unavailable	`{"error": "model_unavailable", "message": "Recommendation model is temporarily unavailable"}`

8.2 Batch Recommendation Endpoint

Endpoint: `GET /recommendations/batch`

Description: Generates recommendations for multiple users concurrently with pagination support. This endpoint is designed for bulk processing scenarios such as nightly batch jobs or analytics pipelines.

Query Parameters

Parameter	Type	Required	Description
`page`	integer	optional	Page number for pagination (default: 1, min: 1)
`limit`	integer	optional	Number of users per page (default: 20, min: 1, max: 100)

Behavioral Requirements

- **Pagination:** Must support efficient pagination through large user sets
- **Concurrency Control:** Implement bounded concurrency to prevent resource exhaustion (you decide the optimal limit)
- **Partial Failure Handling:** Individual user failures must not halt the entire batch
- **Query Optimization:** Must avoid N+1 query problems through batching or eager loading

Response Format (200 OK)

```
{
  "page": 1,
  "limit": 20,
  "total_users": 45,
  "results": [
    {
      "user_id": 1,
      "recommendations": [...],
      "status": "success"
    },
    {
      "user_id": 2,
      "status": "failed",
      "error": "model_inference_timeout",
      "message": "Recommendation generation exceeded timeout limit"
    }
  ],
  "summary": {
    "success_count": 19,
    "failed_count": 1,
    "processing_time_ms": 1250
  },
  "metadata": {
    "generated_at": "2026-02-15T10:30:00Z"
  }
}
```

---

9. Caching Strategy

9.1 Cache Implementation Requirements

- **Cache Key Design:** Use structured keys (e.g., `rec:user:{user_id}:limit:{limit}`)
- **TTL Strategy:** Set appropriate expiration times (recommended: 5-15 minutes)
- **Cache Warming:** Optional pre-population for high-traffic users

- **Invalidation:** Clear cache when user watch history is updated

9.2 Cache Metadata

Include cache hit/miss information in API responses to facilitate debugging and monitoring.

---

10. Performance Testing Requirements

10.1 k6 Test Scenarios

Implement k6 test scripts for the following scenarios:

1. **Single User Load Test:** 100 requests/second for 1 minute
2. **Batch Endpoint Stress Test:** Concurrent batch requests with varying page sizes
3. **Cache Effectiveness Test:** Repeated requests to measure cache hit ratio

10.2 Required Metrics

- **Average Latency:** Mean response time across all requests
- **P95 Latency:** 95th percentile response time
- **P99 Latency:** 99th percentile response time
- **Throughput:** Requests per second successfully processed
- **Error Rate:** Percentage of failed requests

Example k6 Script:

```
import http from 'k6/http';
import { check, sleep } from 'k6';

export let options = {
  stages: [
    { duration: '30s', target: 50},
    { duration: '1m', target: 100},
    { duration: '30s', target: 0 },
  ],
  thresholds: {
    http_req_duration: ['p(95)<500'],
    http_req_failed: ['rate<0.01'],
  },
};

export default function () {
  const userId = Math.floor(Math.random() * 20) + 1;
  const res = http.get(
    `http://localhost:8080/users/${userId}/recommendations?limit=10`
  );

  check(res, {
    'status is 200': (r) => r.status === 200,
    'has recommendations': (r) => JSON.parse(r.body).recommendations.length > 0,
  });

  sleep(0.1);
}
```

---

11. Docker Configuration Requirements

11.1 Docker Compose Setup

Provide a complete `docker-compose.yml` file that includes:

- **Application Service:** Your recommendation service
- **PostgreSQL Service:** Database with persistent volume
- **Redis Service:** Cache layer

Example docker-compose.yml:

```
version: '3.8'

services:
  app:
    build: .
    ports:
      - "8080:8080"
    environment:
      - DATABASE_URL=postgresql://user:password@postgres:5432/recommendations
      - REDIS_URL=redis://redis:6379
    depends_on:
      - postgres
      - redis

  postgres:
    image: postgres:15
    environment:
      - POSTGRES_USER=user
      - POSTGRES_PASSWORD=password
      - POSTGRES_DB=recommendations
    volumes:
      - postgres_data:/var/lib/postgresql/data
    ports:
      - "5432:5432"

  redis:
    image: redis:7
    ports:
      - "6379:6379"

volumes:
  postgres_data:
```

11.2 One-Command Startup

The entire system must be startable with:

```
docker-compose up --build
```

---

12. Database Migration and Seeding

12.1 Migration Scripts

- Implement idempotent migration scripts
- Support both up and down migrations
- Include all required indexes and constraints

12.2 Seed Script

- Must be deterministic (use fixed random seed)
- Should be runnable multiple times (truncate before seeding)
- Must generate realistic data distributions

Example Seed Script Pattern:

```
import random

# Set fixed seed for reproducibility
random.seed(42)

def seed_users(n=20):
    countries = ['US', 'GB', 'CA', 'AU', 'DE']
    subscription_types = ['free', 'basic', 'premium']

    for i in range(n):
        age = random.randint(18, 65)
        country = random.choice(countries)
        subscription = random.choices(
            subscription_types,
            weights=[0.5, 0.3, 0.2]
        )[0]

        # Insert into database
        db.execute(
            "INSERT INTO users (age, country, subscription_type) VALUES (?, ?, ?)",
            (age, country, subscription)
        )
```

---

13. Deliverables

Submit a Git repository containing:

1. **Source Code:** Complete implementation following the specified architecture
2. **Migration Scripts:** Database schema creation and index definitions
3. **Seed Script:** Deterministic data generation
4. **Docker Configuration:** Dockerfile and docker-compose.yml
5. **k6 Test Scripts:** Performance test scenarios
6. **README.md:** Comprehensive documentation

---

14. Documentation Requirements (README.md)

Your README must include:

14.1 Setup Instructions

- Prerequisites and dependencies
- Step-by-step installation guide
- Commands to run migrations and seeding
- How to start the application

14.2 Architecture Overview

- High-level system design diagram or description
- Explanation of each architectural layer
- Data flow through the system
- How the recommendation model integrates with database queries

14.3 Design Decisions

- Caching strategy and TTL rationale
- Concurrency control approach
- Error handling philosophy
- Database indexing strategy
- Scoring algorithm rationale and weight choices

14.4 Performance Results

- k6 test results with metrics
- Identified bottlenecks and limiting factors
- Cache hit rate analysis

14.5 Trade-offs and Future Improvements

- Known limitations of current implementation
- Scalability considerations
- Proposed enhancements if time permitted

---

15. Evaluation Criteria

Your submission will be evaluated on:

Criterion	Weight	Focus Areas
System Architecture	25%	Layer separation, code organization, maintainability
Database Design	15%	Schema correctness, indexing strategy, query efficiency
API Implementation	15%	Endpoint correctness, error handling, response format
Model Integration	15%	Proper use of database data, scoring logic, filtering
Caching Strategy	10%	Implementation effectiveness, cache invalidation
Performance	10%	Latency, throughput, resource utilization
Code Quality	10%	Readability, best practices, documentation

---

16. Submission Guidelines

- **Repository:** Provide a link to a public Git repository (GitHub, GitLab, or Bitbucket)
- **Commit History:** Include meaningful commit messages demonstrating your development process

- **Testing:** Ensure the application runs successfully with `docker-compose up`
- **Timeline:** Submit within the agreed deadline

Submit your development through your contact email whole Git repository as a zip file
Good luck! We look forward to reviewing your implementation.